# Lab 4: Gaussian Processes

Manfred Clase (mancl774)

2026-10-11

## 2.1 Implementing GP Regression

The function `posteriorGP` implements Algorithm 2.1 in Rasmussen and Williams. It uses the Cholesky decomposition of $K + \sigma_n^2 I$ and returns the posterior mean and variance of $f$ at the points in `XStar`. The kernel is passed in as a separate function, so the hyperparameters $\sigma_f$ and $\ell$ are set in the kernel and not in `posteriorGP`. The code is listed in the appendix.

**Theory.** A GP is a prior over functions: any finite set of function values is jointly Gaussian with mean 0 and covariance given by the kernel, $f(x) \sim N(0, K)$. Conditioning on noisy data $y$ gives a Gaussian posterior at test inputs $X_*$ with

$$\bar{f}_* = K_*^T (K + \sigma_n^2 I)^{-1} y, \qquad \operatorname{cov}(f_*) = K_{**} - K_*^T (K + \sigma_n^2 I)^{-1} K_*,$$

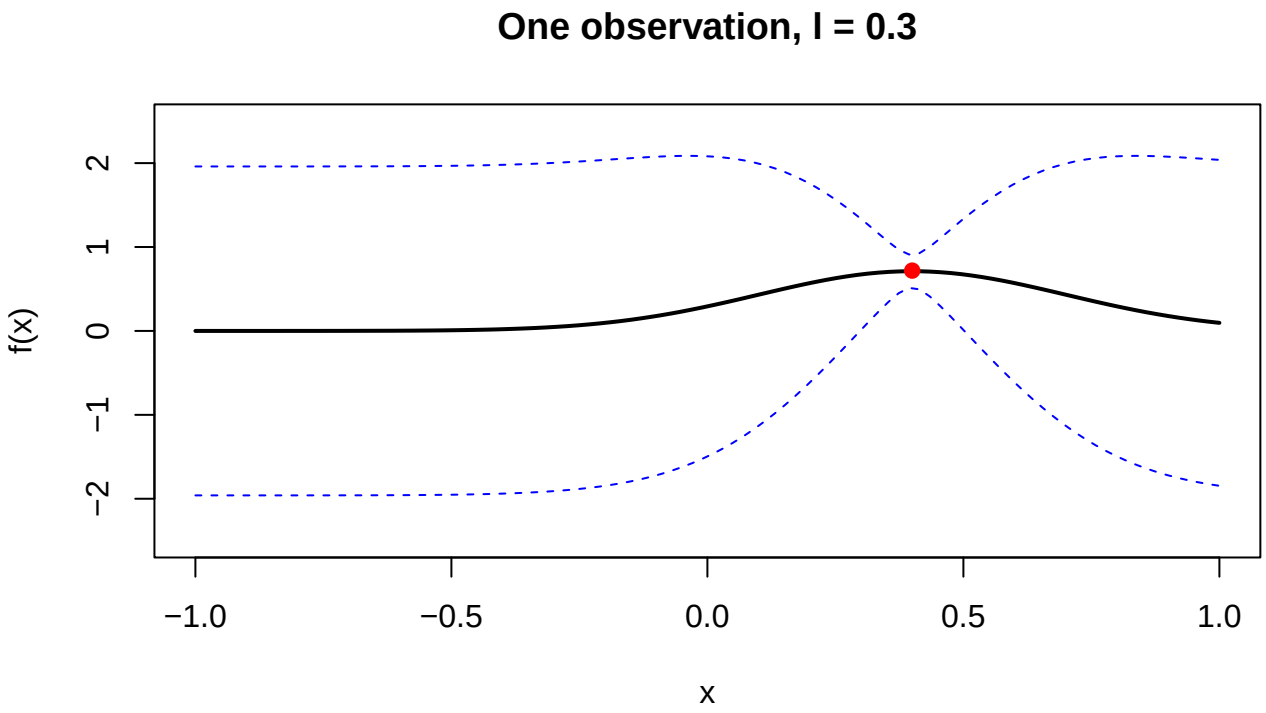
where $K = k(X, X)$, $K_* = k(X, X_*)$ and $K_{**} = k(X_*, X_*)$. The covariance does not depend on $y$, only on where the inputs are: the variance shrinks near observations and goes back to the prior variance $\sigma_f^2$ far from them.

This is ordinary Gaussian conditioning. With $a = f_*$ and $b = y$ jointly Gaussian, $\Sigma_{aa} = K_{**}$, $\Sigma_{ab} = K_*^T$ and $\Sigma_{bb} = K + \sigma_n^2 I$, so $\mu_{a|b} = \Sigma_{ab}\Sigma_{bb}^{-1} b$ and $\Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ give the formulas above. The noise only appears on the diagonal of the training block: $y_i = f(x_i) + \epsilon_i$ with independent noise, so $\operatorname{cov}(y_i, y_j) = k(x_i, x_j) + \sigma_n^2$ only when $i = j$, while $\operatorname{cov}(y, f_*) = K_*$ has no noise term since $f_*$ is noise-free.

As a check, with one observation at $(0.4, 0.719)$ the posterior mean at $x = 0.4$ is $0.719/1.01 \approx 0.712$ and the variance is $1 - 1/1.01 \approx 0.0099$, which matches what the function returns.

### 2.1 (2) One observation

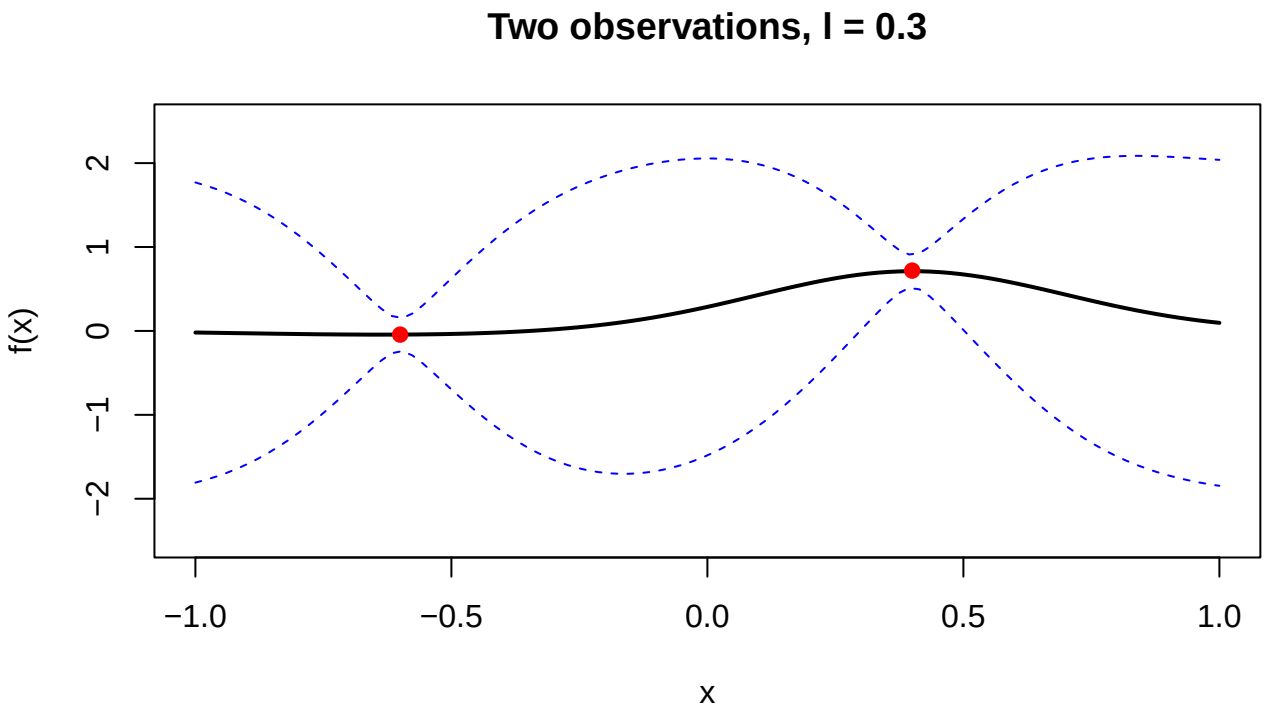
*Implementation:* Ran `posteriorGP` with the single point $(0.4, 0.719)$, $\sigma_n = 0.1$ and the SE kernel with $\sigma_f = 1$, $\ell = 0.3$ on a 100-point grid over $[-1, 1]$; bands are mean $\pm 1.96$ times the posterior sd of $f$.




The mean goes close to the observation and the band is narrow there. Further away the mean goes back to 0 and the band widens to the prior, $\pm 1.96$, since the point has no influence more than about $3\ell$ away.

### 2.1 (3) Two observations

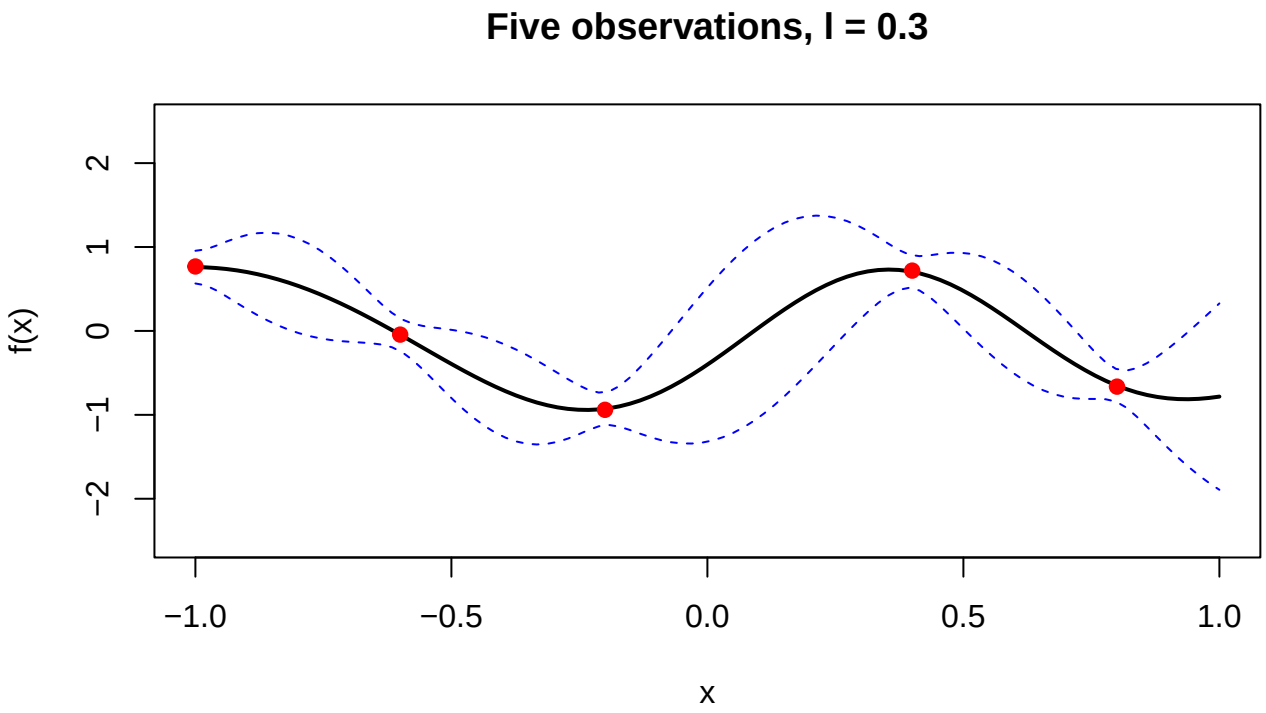
*Implementation:* Recomputed the posterior from scratch on both points, $(0.4, 0.719)$ and $(-0.6, -0.044)$, with the same $\sigma_n = 0.1$, $\sigma_f = 1$, $\ell = 0.3$ and grid.




The new point only changes the posterior near itself. The two points are 1 apart (more than $3\ell$), so they are almost uncorrelated and the curve around $x = 0.4$ is the same as before.

### 2.1 (4) Five observations

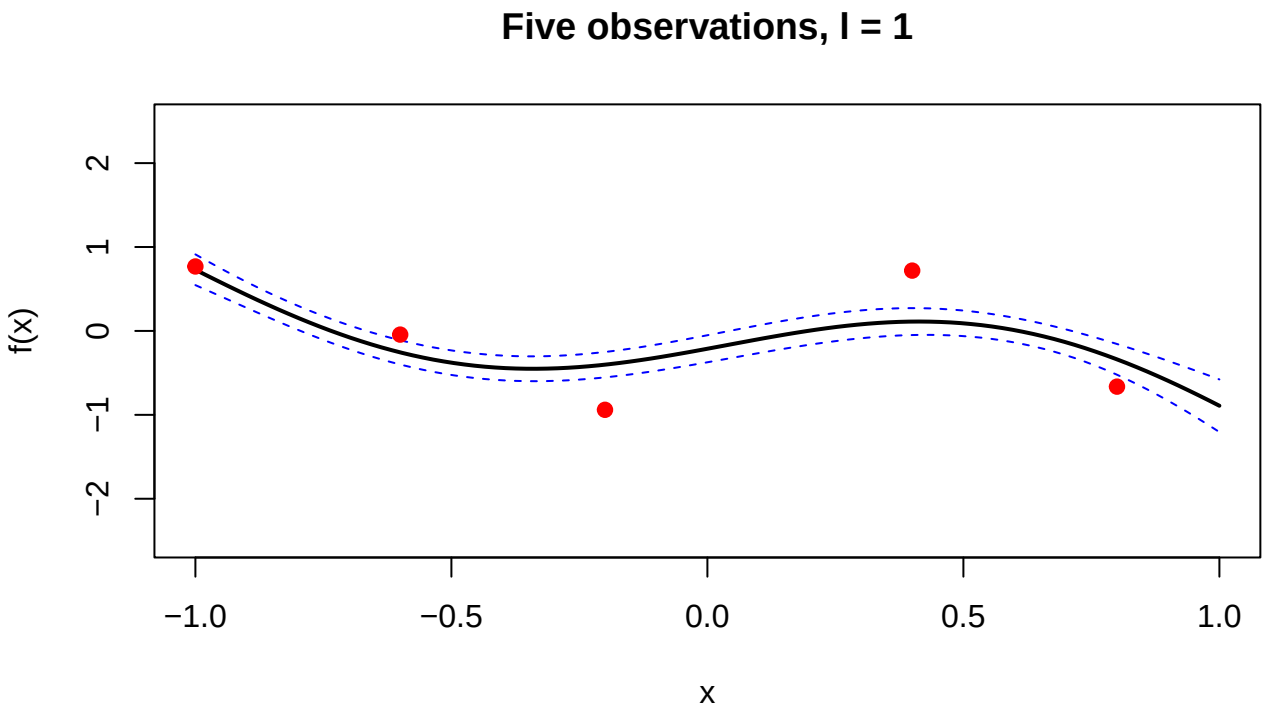
*Implementation:* Same call on all five given points, still $\sigma_n = 0.1$, $\sigma_f = 1$, $\ell = 0.3$.




With five points the mean follows the data closely and the band is narrow wherever there is data nearby. It only gets wider in the gaps between points.

### 2.1 (5) Five observations, $\ell = 1$

*Implementation:* Same five points and $\sigma_n = 0.1$, $\sigma_f = 1$, only the length scale changed to $\ell = 1$.




With $\ell = 1$ the prior assumes a much smoother function. Each point now affects a large part of the interval, so the bands are narrower, but the mean is too stiff to follow the data, which goes up and down a lot over short distances. It misses most of the points. For this data $\ell = 0.3$ gives a much better fit.

In general, $\ell$ sets how fast the correlation between $f(x)$ and $f(x')$ decays with distance: a small $\ell$ gives wiggly functions that can overfit, a large $\ell$ gives smooth functions that can underfit. $\sigma_f$ sets the vertical scale (how far $f$ typically goes from the mean) and $\sigma_n$ how much of the variation in $y$ is treated as noise. Here the hyperparameters are fixed by hand; in practice they are usually chosen by maximizing the log marginal likelihood $\log p(y \mid X, \sigma_f, \ell, \sigma_n)$, which balances data fit against model complexity.

## 2.2 GP Regression with kernlab

Only every fifth day is used because exact GP regression factorizes the $n \times n$ matrix $K + \sigma_n^2 I$, which costs $O(n^3)$ time: using all 2190 days would be about $5^3 = 125$ times slower than using 438.

### 2.2 (1) Own SE kernel

The kernel is written in the kernlab format: an outer function that takes $\sigma_f$ and $\ell$ and returns the kernel function, labeled with class `"kernel"`. With $\sigma_f = 1$ and $\ell = 1$, the kernel at $x = 1$, $x' = 2$ is:

```
## [1] 0.6065307
```

and the covariance matrix $K(X, X_*)$ for $X = (1, 3, 4)^T$ and $X_* = (2, 3, 4)^T$ is:

```
## An object of class "kernelMatrix"
##        [,1]   [,2]   [,3]
## [1,] 0.6065 0.1353 0.0111
## [2,] 0.6065 1.0000 0.6065
## [3,] 0.1353 0.6065 1.0000
```

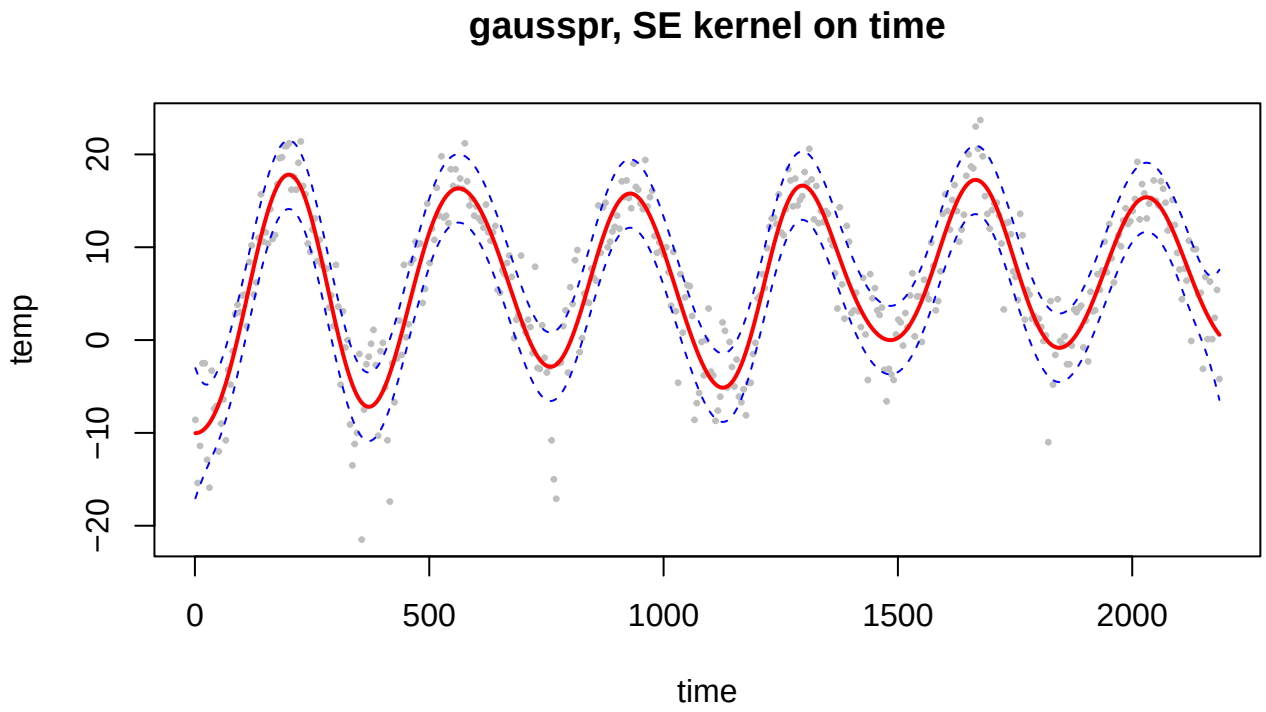
The entries are 1 where the inputs are equal and get smaller with distance.

### 2.2 (2) Model with time

*Implementation:* Fitted `gausspr` on time (every 5th day, 438 points) with the own SE kernel ($\sigma_f = 20$, $\ell = 100$), `var` = residual variance of the quadratic `lm` fit, `scaled = FALSE` and `variance.model = TRUE`; plotted the posterior mean and $\pm 1.96$ sd bands at the training inputs.

The noise variance $\sigma_n^2$ is the residual variance from a quadratic regression of temperature on time:

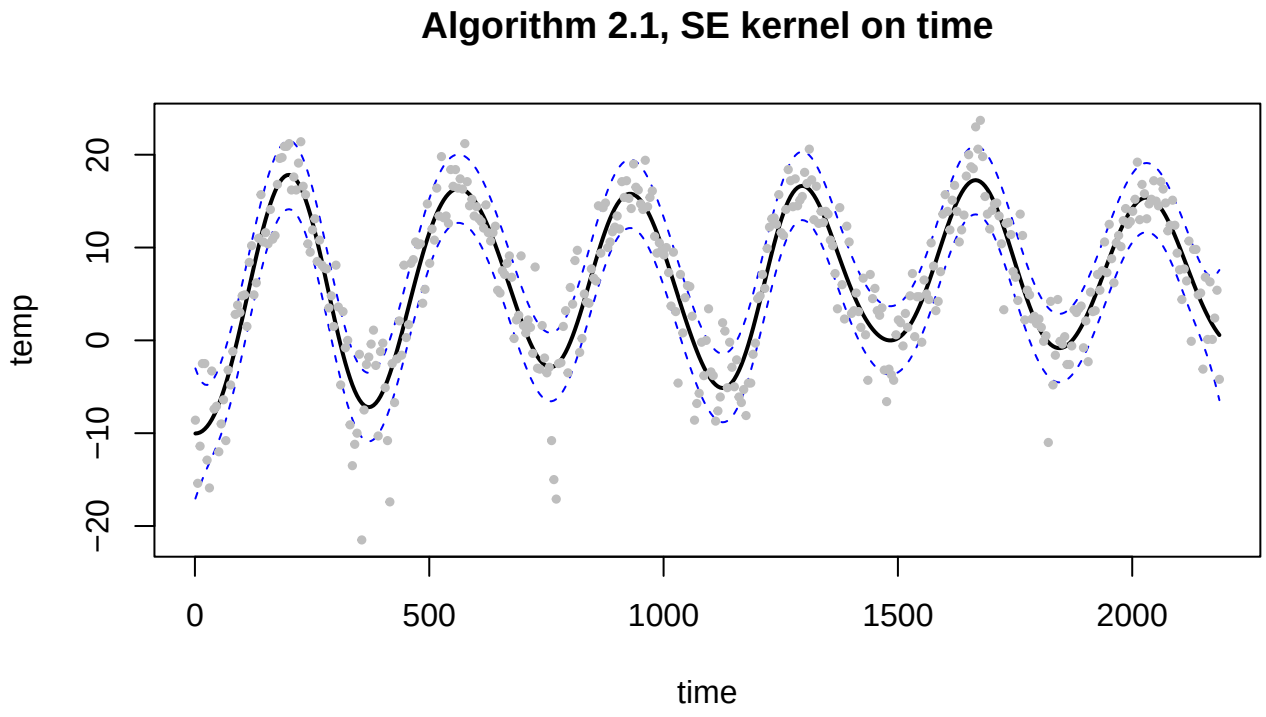
```
## [1] 66.85169
```




The variance is quite large (sd about 8 degrees) because a quadratic over six years cannot follow the seasons, so the residuals include the seasonal swing as well as the day-to-day weather. The posterior mean follows the yearly cycle smoothly. With $\ell = 100$ days it picks up the seasons but not the short-term variation.

### 2.2 (3) Same model with Algorithm 2.1

*Implementation:* Ran `posteriorGP` on the same 438 points with the own `SquaredExpKernel` ($\sigma_f = 20$, $\ell = 100$) and noise sd $\sigma_n$ (the square root of the residual variance $\sigma_n^2$ from 2.2 (2)), predicting at the training times; bands use the posterior variance of $f$ only.

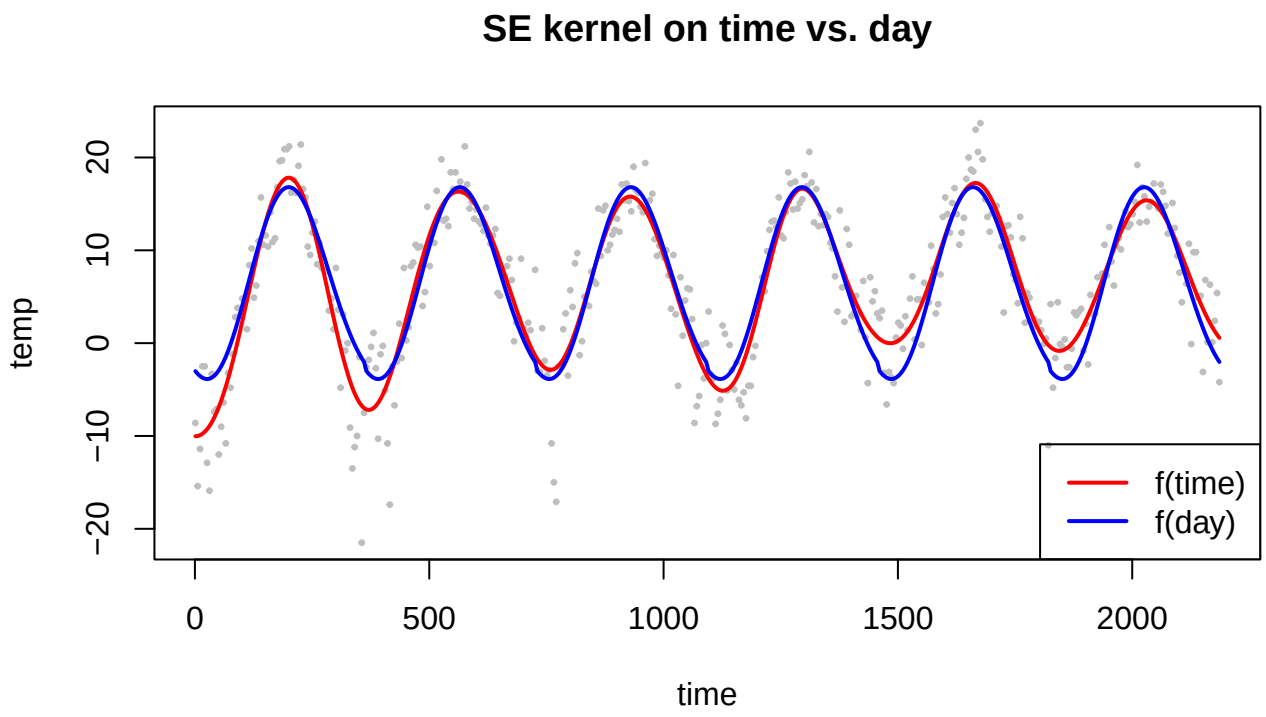



The posterior mean and the bands are the same as from `gausspr`, which confirms that `posteriorGP` is correct. Many points are outside the bands, but that is expected. The bands are for $f$, the smooth underlying temperature, and not for the individual measurements, which scatter around $f$ with the noise. A band for new observations would be mean $\pm\, 1.96\sqrt{\text{var} + \sigma_n^2}$:

```
# Bands for new observations y* = f* + noise: add the noise variance.
# (sigmaNoise from 2.2 (2) holds the noise VARIANCE.) Run after plotGP.
sdY <- sqrt(res$var + sigmaNoise)
lines(time, res$mean + 1.96 * sdY, col = "darkgreen", lty = 3)
lines(time, res$mean - 1.96 * sdY, col = "darkgreen", lty = 3)
```

## 2.2 (4) Model with day

*Implementation:* Fitted `gausspr` with the same kernel ($\sigma_f = 20$, $\ell = 100$), noise variance and `scaled = FALSE`, but with day of year as input; its mean is plotted against time together with the time model's mean.

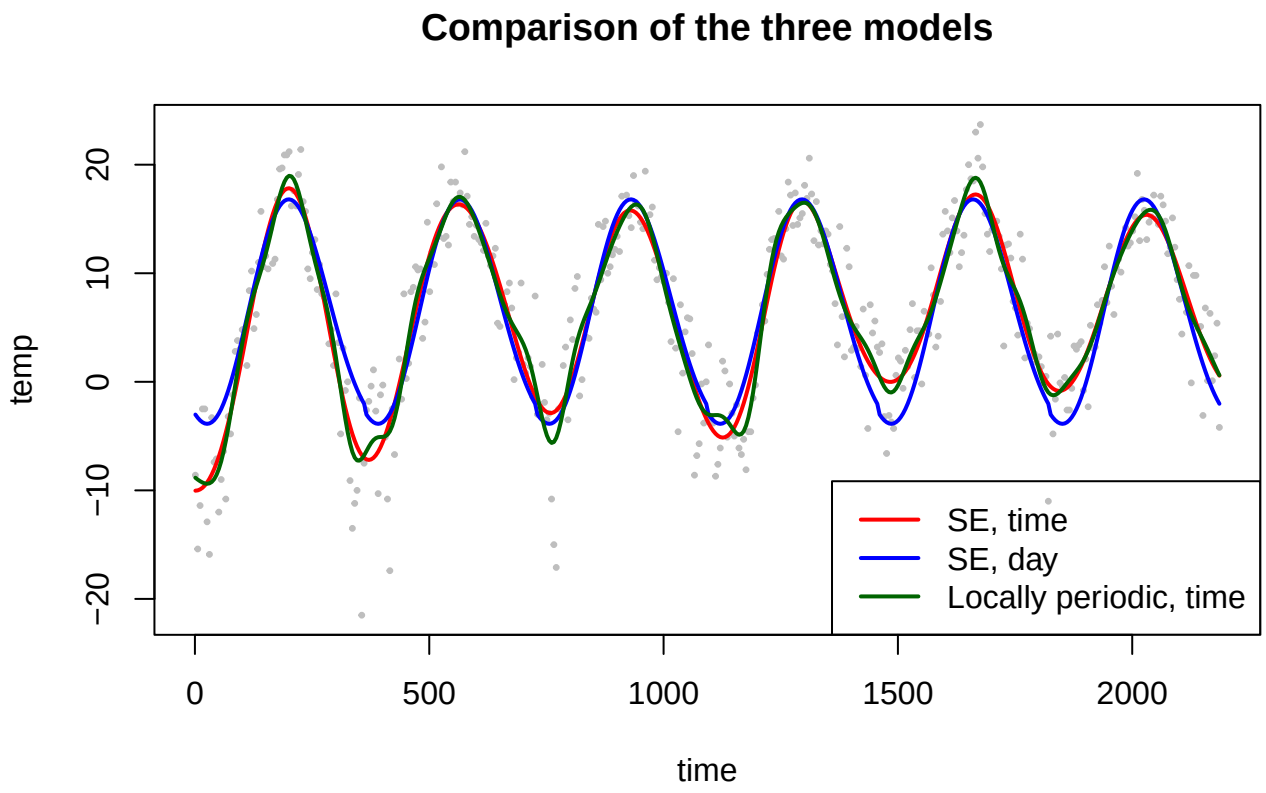



The day model gives the same curve every year. All six measurements of the same date get the same input, so the model pools them into one average seasonal profile (winters around −4, summers around 17).

**Pros and cons.** The day model uses all six years to estimate each day of the year, so the seasonal pattern is stable and it can be used to predict future years. But it cannot capture differences between years, like the cold winters at the start and in 2010/11 or the mild winter around time 1500. The year-to-year variation is just treated as noise. It also treats 31 December and 1 January as far apart (distance 364), even though they are neighbours. In general, the choice of input and kernel encodes assumptions: with day as input $f$ is forced to be exactly periodic with period 365. The time model follows each year's actual data, but it does not know that the pattern repeats. Each year is only learned from nearby data, and outside the data it would go back to the prior mean 0.

## 2.2 (5) Locally periodic kernel

*Implementation:* Wrote a kernlab-style locally periodic kernel and fitted `gausspr` on time with $\sigma_f = 20$, $\ell_1 = 1$, $\ell_2 = 100$, $d = 365$, the same noise variance and `scaled = FALSE`; plotted its posterior mean against the two SE models.




The locally periodic model follows the data more closely than the other two. It has higher summer peaks, lower winter lows, and some small bumps that the SE models smooth out (around time 400, 750 and 1100).

This is because the kernel is less smooth for nearby points. For small distances the periodic part with $\ell_1 = 1$ behaves like an SE kernel with length scale $d/(2\pi) \approx 58$ days, and multiplied with the SE part ($\ell_2 = 100$) the combined length scale is about 50 days, half of the 100 used in the other models. The bumps show up mostly in winter because the winter data is much more spread out, with cold snaps down to $-15$ to $-20$, and the more flexible model bends toward them. Summer data is more tightly clustered, so the curve stays smooth there.

The yearly part does not do much with these hyperparameters. One year apart, the SE factor with $\ell_2 = 100$ is almost zero ($k(1, 366) \approx 0.5$ compared to 400 at distance 0), so the years barely share information. The result is mainly a more flexible version of the time model, which still captures differences between years, unlike the day model. A larger $\ell_2$ would let the periodic part carry information between years and combine the strengths of both earlier models.

In general, multiplying kernels gives a function with both properties: the periodic factor makes points one period apart similar, and the SE factor lets that similarity fade with distance, so the seasonal pattern can drift slowly over the years. For points exactly $n$ periods apart the periodic factor is 1, so $k(x, x + nd) = \sigma_f^2 \exp(-(nd)^2/(2\ell_2^2))$, and $\ell_2$ alone decides how much one year says about the next.

## 2.3 GP Classification with kernlab

### 2.3 (1) Two covariates

*Implementation:* Fitted on the 1000 training rows (`set.seed(111)`), training accuracy from the confusion matrix of predicted classes; the contour shows $P(\text{fraud} = 1)$ on a $100 \times 100$ grid over the training range of `varWave` and `skewWave`.

The model uses the default kernel (RBF, with $\sigma$ chosen automatically by `sigest`) and only `varWave` and `skewWave`.

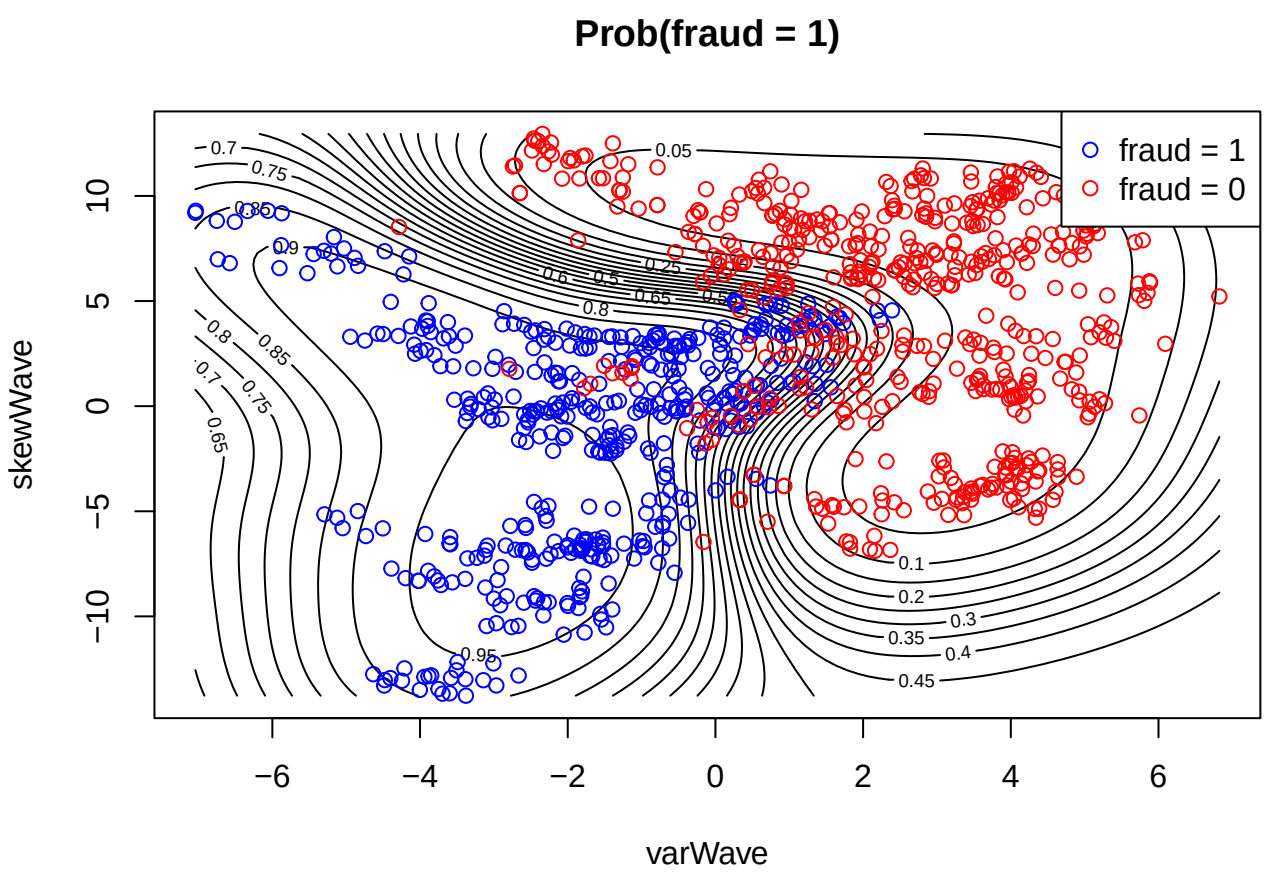
**Theory.** In GP classification a GP prior is put on a latent function $f(x)$, which is squashed through a link function (logistic or probit) to give $P(y = 1 \mid x) = s(f(x))$, with $s$ the link. The likelihood is then not Gaussian, so the posterior has no closed form and is approximated (e.g. Laplace approximation or expectation propagation). The output is a class probability, and the decision boundary ($P = 0.5$) can be nonlinear since $f$ is a flexible function of the inputs.

```
## Using automatic sigma estimation (sigest) for RBF or laplace kernel
```

```
##        predicted
## actual   0   1
##      0 503  41
##      1  18 438
```

```
## [1] 0.941
```

The training accuracy is 0.941.




The probability of fraud is high (about 0.9) around the fraud points on the left and low (about 0.05) around the genuine notes on the right. The decision boundary is near `varWave` = 0, where the two classes overlap, which is where the errors come from. Far away from the data the probabilities move back toward 0.5, since the GP goes back to its prior where there is no information.

### 2.3 (2) Test set

*Implementation:* Used the same two-covariate model to predict classes for the 372 held-out test notes; accuracy is the diagonal of the confusion matrix over its total.

```
##        predicted
## actual   0   1
##      0 199  19
##      1   9 145
```

```
## [1] 0.9247312
```

The test accuracy is 0.925, slightly lower than on the training data, as expected. The difference is small, so the model is not overfitting much.

## 2.3 (3) All four covariates

*Implementation:* Refitted `gausspr` with `fraud ~ .` (all four covariates, default RBF kernel) on the same 1000 training rows and computed the confusion matrix and accuracy on the 372 test notes.

```
## Using automatic sigma estimation (sigest) for RBF or laplace kernel
```

```
##       predicted
## actual   0   1
##      0 216   2
##      1   0 154
```

```
## [1] 0.9946237
```

With all four covariates the test accuracy is 0.995, compared to 0.925 with two. Only 2 of the 372 test notes are misclassified. `kurtWave` and `entropyWave` contain information that separates the classes much better, especially in the region where the two classes overlap when only `varWave` and `skewWave` are used.

## Appendix: R Code

```
knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE,
                      fig.width = 7, fig.height = 4.2, fig.align = "center")
library(kernlab)
# Squared exponential kernel (from GaussianProcesses.R on the course page)
# k(x, x') = sigmaF^2 * exp(-(x - x')^2 / (2 l^2))
#   sigmaF: vertical scale (prior sd of f, k(x, x) = sigmaF^2)
#   l:      length scale, larger l = smoother f (corr. reach ~ 3l)
# Returns the n1 x n2 covariance matrix K(x1, x2), filled column by column.
SquaredExpKernel <- function(x1, x2, sigmaF = 1, l = 3) {
  n1 <- length(x1)
  n2 <- length(x2)
  K <- matrix(NA, n1, n2)
  for (i in 1:n2) {
    K[, i] <- sigmaF^2 * exp(-0.5 * ((x1 - x2[i]) / l)^2)
  }
  return(K)
}

# Algorithm 2.1 (Rasmussen and Williams): posterior of f at XStar.
# Cholesky instead of solve(K): numerically stable (K is often near
# singular) and the O(n^3) factorisation is done once; rest is triangular.
# Note: sigmaNoise is the noise SD, it is squared here.
posteriorGP <- function(X, y, XStar, sigmaNoise, k) {
  K <- k(X, X)
  # chol() returns upper triangular R (K = R^T R), so t() gives lower L
  L <- t(chol(K + sigmaNoise^2 * diag(nrow(K))))
  # alpha = (K + sigmaN^2 I)^-1 y = L^T \ (L \ y)
  alpha <- solve(t(L), solve(L, y))
  # K* = K(X, XStar), n x n*
  k_star <- k(X, XStar)
  # Posterior mean: K*^T alpha
  f_star_mean <- t(k_star) %*% alpha
  # v = L \ K*, so v^T v = K*^T (K + sigmaN^2 I)^-1 K*
  v <- solve(L, k_star)
  # Posterior variance: diag(K**) - diag(v^T v) (= colSums(v^2), faster)
  f_star_var <- diag(k(XStar, XStar)) - diag(t(v) %*% v)
  return(list(mean = f_star_mean, var = f_star_var))
}

# Plot posterior mean with 95% pointwise bands: mean +- 1.96 * sd.
# These are bands for f (not for new y; for y add sigmaN^2 to var).
plotGP <- function(X, y, XStar, res, main = "", ylim = c(-2.5, 2.5),
                   xlab = "x", ylab = "f(x)", ptcol = "red", ptcex = 1) {
  sd <- sqrt(res$var)
  plot(XStar, res$mean, type = "l", lwd = 2, ylim = ylim,
       xlab = xlab, ylab = ylab, main = main)
  lines(XStar, res$mean + 1.96 * sd, col = "blue", lty = 2)
  lines(XStar, res$mean - 1.96 * sd, col = "blue", lty = 2)
  points(X, y, pch = 19, col = ptcol, cex = ptcex)
}

# Prediction grid on [-1, 1]
```

```r
XStar <- seq(-1, 1, length.out = 100)
# Fix hyperparameters in closures so posteriorGP only sees k(x1, x2)
k03 <- function(x1, x2) SquaredExpKernel(x1, x2, sigmaF = 1, l = 0.3)
k1  <- function(x1, x2) SquaredExpKernel(x1, x2, sigmaF = 1, l = 1)
# One observation, sigmaN = 0.1, l = 0.3
X <- 0.4; y <- 0.719
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k03)
plotGP(X, y, XStar, res, main = "One observation, l = 0.3")
# Add a second observation; posterior recomputed from scratch on both
X <- c(0.4, -0.6); y <- c(0.719, -0.044)
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k03)
plotGP(X, y, XStar, res, main = "Two observations, l = 0.3")
# All five observations, still l = 0.3
X <- c(-1.0, -0.6, -0.2, 0.4, 0.8)
y <- c(0.768, -0.044, -0.940, 0.719, -0.664)
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k03)
plotGP(X, y, XStar, res, main = "Five observations, l = 0.3")
# Same five points, l = 1: smoother prior, stiffer mean, narrower bands
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k1)
plotGP(X, y, XStar, res, main = "Five observations, l = 1")
# Tullinge daily temperatures, 2190 days = 6 years
# read.csv straight from the URL; file has a header and is ";"-separated
data <- read.csv(
↪  "https://github.com/STIMALiU/AdvMLCourse/raw/master/GaussianProcess/Code/TempTullinge.csv"
↪  ,
                 header = TRUE, sep = ";")
# time: day index 1..2190; day: day of year 1..365 (repeats each year)
time <- 1:2190
day <- rep(1:365, 6)
# Keep every 5th observation (438 points) so the O(n^3) fits are fast
idx <- seq(1, 2190, by = 5)
time <- time[idx]; day <- day[idx]; temp <- data$temp[idx]
# kernlab-style SE kernel: outer function fixes the hyperparameters and
# returns k(x, y) for ONE pair of points (kernlab loops over pairs).
# Unlike rbfdot, this has sigmaf (kernlab's SE has no vertical scale).
SEkernelKL <- function(sigmaf = 1, ell = 1) {
  rval <- function(x, y = NULL) {
    # Euclidean distance, works for vector inputs too
    r <- sqrt(sum((x - y)^2))
    return(sigmaf^2 * exp(-(r^2 / (2 * ell^2))))
  }
  # Class "kernel" makes gausspr/kernelMatrix accept it as a kernel
  class(rval) <- "kernel"
  return(rval)
}

kern <- SEkernelKL(sigmaf = 1, ell = 1)
# k(1, 2) = exp(-1/2) = 0.6065
kern(1, 2)
# kernelMatrix(kernel, x, y) = K(x, y), rows x, cols y (returns class
# "kernelMatrix", as.matrix for printing)
round(as.matrix(kernelMatrix(kern, c(1, 3, 4), c(2, 3, 4))), 4)
# sigmaN^2 = residual variance of a quadratic fit in time.
# I() is needed so time^2 is a real squared term in the formula.
```

```r
fit <- lm(temp ~ time + I(time^2))
sigmaNoise <- var(fit$residuals)   # variance, not sd
sigmaNoise

# Given hyperparameters: sigmaf = 20 (degrees), ell = 100 (days)
kern_20_100 <- SEkernelKL(sigmaf = 20, ell = 100)
# gausspr args:
#   kernel = kernel object (or a generator + kpar = list(...))
#   var    = noise VARIANCE sigmaN^2 (not sd)
#   variance.model = TRUE needed for type = "sdeviation" later
#   scaled = FALSE: don't standardise x/y, so ell and sigmaf are on the
#            data scale (days, degrees)
GPfit <- gausspr(time, temp, kernel = kern_20_100, var = sigmaNoise,
                 variance.model = TRUE, scaled = FALSE)
# Posterior mean at the training inputs
meanPred <- predict(GPfit, time)
# Posterior sd of f (only available because variance.model = TRUE)
sdPred <- predict(GPfit, time, type = "sdeviation")

# Data, posterior mean and 95% bands (mean +- 1.96 sd)
plot(time, temp, pch = 20, cex = 0.5, col = "grey", main = "gausspr, SE kernel on time")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPred + 1.96 * sdPred, col = "blue", lty = 2)
lines(time, meanPred - 1.96 * sdPred, col = "blue", lty = 2)
# Same model via own Algorithm 2.1, should match gausspr exactly
k_20_100 <- function(x1, x2) SquaredExpKernel(x1, x2, sigmaF = 20, l = 100)
# posteriorGP wants the noise SD, so sqrt of the variance from above
res <- posteriorGP(time, temp, time, sigmaNoise = sqrt(sigmaNoise), k = k_20_100)
plotGP(time, temp, time, res, main = "Algorithm 2.1, SE kernel on time",
       ylim = range(temp), xlab = "time", ylab = "temp", ptcol = "grey", ptcex = 0.5)
# Bands for new observations y* = f* + noise: add the noise variance.
# (sigmaNoise from 2.2 (2) holds the noise VARIANCE.) Run after plotGP.
sdY <- sqrt(res$var + sigmaNoise)
lines(time, res$mean + 1.96 * sdY, col = "darkgreen", lty = 3)
lines(time, res$mean - 1.96 * sdY, col = "darkgreen", lty = 3)
# Same kernel and noise, but input = day of year: all years share one
# seasonal curve
GPfit_day <- gausspr(day, temp, kernel = kern_20_100, var = sigmaNoise,
                     variance.model = TRUE, scaled = FALSE)
meanPred_day <- predict(GPfit_day, day)

# Plot both against time to compare (day model repeats every year)
plot(time, temp, pch = 20, cex = 0.5, col = "grey", main = "SE kernel on time vs. day")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPred_day, col = "blue", lwd = 2)
legend("bottomright", legend = c("f(time)", "f(day)"), col = c("red", "blue"), lwd = 2)
# Locally periodic kernel:
# k(x, x') = sigmaf^2 * exp(-2 sin^2(pi r / d) / ell1^2)
#                     * exp(-r^2 / (2 ell2^2)),       r = |x - x'|
#   sigmaf: vertical scale
#   d:      period (365 days), points d apart are "the same season"
#   ell1:   smoothness within a period (periodic part)
#   ell2:   how fast correlation between periods decays (SE part);
#           large ell2 = years share more information
```

```r
SELPkernelKL <- function(sigmaf = 1, ell1 = 1, ell2 = 1, d = 365) {
  rval <- function(x, y = NULL) {
    r <- abs(x - y)
    # periodic factor * SE factor
    return(sigmaf^2 * exp(-2 * sin(pi * r / d)^2 / ell1^2) * exp(-0.5 * r^2 / ell2^2))
  }
  # Needed so kernlab treats it as a kernel
  class(rval) <- "kernel"
  return(rval)
}

# Fit on time with the same noise variance and data-scale settings
kern_lp <- SELPkernelKL(sigmaf = 20, ell1 = 1, ell2 = 100, d = 365)
GPfit_lp <- gausspr(time, temp, kernel = kern_lp, var = sigmaNoise,
                    variance.model = TRUE, scaled = FALSE)
meanPred_lp <- predict(GPfit_lp, time)

plot(time, temp, pch = 20, cex = 0.5, col = "grey", main = "Comparison of the three
↪  models")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPred_day, col = "blue", lwd = 2)
lines(time, meanPred_lp, col = "darkgreen", lwd = 2)
legend("bottomright", legend = c("SE, time", "SE, day", "Locally periodic, time"),
       col = c("red", "blue", "darkgreen"), lwd = 2)
# Banknote data: 4 wavelet features + fraud label (0/1)
# File has no header row (header = FALSE), so columns are named below
data <- read.csv(
↪  "https://github.com/STIMALiU/AdvMLCourse/raw/master/GaussianProcess/Code/banknoteFraud.csv"
↪  ,
                 header = FALSE, sep = ",")
names(data) <- c("varWave", "skewWave", "kurtWave", "entropyWave", "fraud")
# Factor response -> gausspr does classification (not regression)
data[, 5] <- as.factor(data[, 5])

# Given split: seed 111, 1000 random rows for training, rest (372) test
set.seed(111)
SelectTraining <- sample(1:dim(data)[1], size = 1000, replace = FALSE)
train <- data[SelectTraining, ]
test <- data[-SelectTraining, ]
# Formula interface, default kernel rbfdot with sigma from sigest()
GPfitFraud <- gausspr(fraud ~ varWave + skewWave, data = train)

# Default predict type = "response" gives the predicted class labels
pred_train <- predict(GPfitFraud, train[, c("varWave", "skewWave")])
# Confusion matrix: rows = truth, cols = prediction
cm_train <- table(actual = train$fraud, predicted = pred_train)
# Accuracy = correct (diagonal) / total
accuracy_train <- sum(diag(cm_train)) / sum(cm_train)
cm_train
accuracy_train
# 100 x 100 grid over the range of the two covariates
x1 <- seq(min(train$varWave), max(train$varWave), length = 100)
x2 <- seq(min(train$skewWave), max(train$skewWave), length = 100)
# expand.grid varies x1 fastest; column names must match the formula
```

```r
gridPoints <- expand.grid(varWave = x1, skewWave = x2)
# Class probabilities, one column per level ("0", "1")
probPreds <- predict(GPfitFraud, gridPoints, type = "probabilities")

# x1-fastest order fills the matrix column-wise as contour expects
# (z[i, j] at x1[i], x2[j]), so no byrow = TRUE here. 20 = n levels.
contour(x1, x2, matrix(probPreds[, 2], 100, 100), 20,
        xlab = "varWave", ylab = "skewWave", main = "Prob(fraud = 1)")
# Overlay training points by class
points(train$varWave[train$fraud == 1], train$skewWave[train$fraud == 1], col = "blue")
points(train$varWave[train$fraud == 0], train$skewWave[train$fraud == 0], col = "red")
legend("topright", legend = c("fraud = 1", "fraud = 0"),
       col = c("blue", "red"), pch = 1, bg = "white")
# Same model, evaluated on the 372 held-out notes
pred_test <- predict(GPfitFraud, test[, c("varWave", "skewWave")])
cm_test <- table(actual = test$fraud, predicted = pred_test)
accuracy_test <- sum(diag(cm_test)) / sum(cm_test)
cm_test
accuracy_test
# fraud ~ . uses all four covariates
GPfitFraud_4 <- gausspr(fraud ~ ., data = train)

pred_test_4 <- predict(GPfitFraud_4, test[, c("varWave", "skewWave", "kurtWave",
↪  "entropyWave")])
cm_test_4 <- table(actual = test$fraud, predicted = pred_test_4)
accuracy_test_4 <- sum(diag(cm_test_4)) / sum(cm_test_4)
cm_test_4
accuracy_test_4
ell <- 2
SE <- rbfdot(sigma = 1 / (2 * ell^2))      # SE kernel, no sigmaf
Matern32 <- function(sigmaf = 1, ell = 1) {
  rval <- function(x, y = NULL) {
    r <- sqrt(crossprod(x - y))   # crossprod(v) = t(v) %*% v, 1x1 matrix
    sigmaf^2 * (1 + sqrt(3) * r / ell) * exp(-sqrt(3) * r / ell)
  }
  class(rval) <- "kernel"   # lets gausspr/kernelMatrix use it as a kernel
  rval
}
# kpar: named args passed to the generator Matern32 to build the kernel
fit <- gausspr(x, y, kernel = Matern32, kpar = list(sigmaf = 1, ell = 2),
               var = sigmaN^2, scaled = FALSE)
# equivalent: kernel = Matern32(sigmaf = 1, ell = 2)
mu <- predict(fit, xs)   # posterior mean only
# Own posterior variance of f at xs (mean is t(Kxs) %*% solve(A, y))
A <- kernelMatrix(SE, x, x) + sigmaN^2 * diag(length(x))
Kxs <- kernelMatrix(SE, x, xs)                    # K(x, xs)
varf <- diag(kernelMatrix(SE, xs, xs) - t(Kxs) %*% solve(A, Kxs))
# Classification: probabilities on a grid
fitC <- gausspr(cls ~ a + b, data = train)   # factor cls -> classification
g <- expand.grid(a = ga, b = gb)   # names must match formula; a varies fastest
p <- predict(fitC, g, type = "probabilities")   # one column per class
# z[i, j] at (ga[i], gb[j]); p[, 2] = P(2nd class); 20 = number of levels
contour(ga, gb, matrix(p[, 2], length(ga)), 20)
```

## Notes: kernlab (from KernLabDemo.R)

- Full demo: `KernLabDemo.R` on the course website.
- `rbfdot(sigma)` is kernlab's SE kernel $\exp(-\sigma r^2)$, so $\sigma = 1/(2\ell^2)$. It has no $\sigma_f$; write own kernel for that.
- Own kernel: generator returns `function(x, y = NULL)` for one pair, with `class(rval) <- "kernel"`. Pass the object, or the generator + `kpar` (list names must match the generator's arguments).
- `gausspr` args: `x`, `y` (or formula + `data`), `kernel`, `kpar`, `var` = noise variance $\sigma_n^2$ (not sd), `scaled = FALSE` keeps hyperparameters on the data scale, `variance.model = TRUE` for sd. `type` ("regression"/"classification") is inferred: factor y = classif.
- Gotcha: `predict` gives the mean only. `type = "sdeviation"` needs `variance.model = TRUE`; otherwise (or for the full covariance, or prediction bands for y) compute it yourself with Alg. 2.1 (`posteriorGP`) or kernelMatrix + solve as below.
- Classification: `predict(..., type = "probabilities")` gives one column per class; contour over an `expand.grid` grid (see 2.3).

```
ell <- 2
SE <- rbfdot(sigma = 1 / (2 * ell^2))      # SE kernel, no sigmaf
Matern32 <- function(sigmaf = 1, ell = 1) {
  rval <- function(x, y = NULL) {
    r <- sqrt(crossprod(x - y))   # crossprod(v) = t(v) %*% v, 1x1 matrix
    sigmaf^2 * (1 + sqrt(3) * r / ell) * exp(-sqrt(3) * r / ell)
  }
  class(rval) <- "kernel"   # lets gausspr/kernelMatrix use it as a kernel
  rval
}
# kpar: named args passed to the generator Matern32 to build the kernel
fit <- gausspr(x, y, kernel = Matern32, kpar = list(sigmaf = 1, ell = 2),
               var = sigmaN^2, scaled = FALSE)
# equivalent: kernel = Matern32(sigmaf = 1, ell = 2)
mu <- predict(fit, xs)   # posterior mean only
# Own posterior variance of f at xs (mean is t(Kxs) %*% solve(A, y))
A <- kernelMatrix(SE, x, x) + sigmaN^2 * diag(length(x))
Kxs <- kernelMatrix(SE, x, xs)                    # K(x, xs)
varf <- diag(kernelMatrix(SE, xs, xs) - t(Kxs) %*% solve(A, Kxs))
# Classification: probabilities on a grid
fitC <- gausspr(cls ~ a + b, data = train)   # factor cls -> classification
g <- expand.grid(a = ga, b = gb)   # names must match formula; a varies fastest
p <- predict(fitC, g, type = "probabilities")   # one column per class
# z[i, j] at (ga[i], gb[j]); p[, 2] = P(2nd class); 20 = number of levels
contour(ga, gb, matrix(p[, 2], length(ga)), 20)
```